Задача за раницата

```
функция knapsack(W, weights, values, n, multiple_items_allowed):
    dp ≔ [[0 за w от 0 до W + 1 прави] за i от 0 до n + 1 прави]

    за i от 1 до n + 1 прави:
        за w от 1 до W + 1 прави:
            ако weights[i-1] ≤ w то:
                ако multiple_items_allowed то:
                    dp[i][w] ≔ max(dp[i-1][w], values[i-1] + dp[i][w-weights[i-1]])
                иначе:
                    dp[i][w] ≔ max(dp[i-1][w], values[i-1] + dp[i-1][w-weights[i-1]])
            иначе:
                dp[i][w] ≔ dp[i-1][w]

    res ≔ dp[n][W]
    w ≔ W
    items_selected ≔ []

    i ≔ n
    докато i > 0 и res > 0 прави:
        # Този продукт е добавен в раницата
        ако multiple_items_allowed то:
            # Проверяваме колко пъти продуктът е добавен, когато многократното добавяне е разрешено
            докато i > 0 и dp[i][w] = dp[i][w - weights[i-1]] + values[i-1] прави:
                items_selected.добави(i)
                w ≔ w - weights[i-1]
                res ≔ res - values[i-1]
```

```
        i ≔ i - 1
    иначе:
        # Когато многократното добавяне не е разрешено, връщаме назад, за да намерим продуктите
        ако dp[i][w] ≠ dp[i-1][w] то:
            items_selected.добави(i)
            res ≔ res - values[i-1]
            w ≔ w - weights[i-1]
        i ≔ i - 1

  items_count ≔ {}
  за item от items_selected прави:
    ако item от items_count то:
        items_count[item] ≔ items_count[item] + 1
    иначе:
        items_count[item] ≔ 1

  върни dp[n][W], items_count

W ≔ 600
weights ≔ [150, 160, 220, 120, 80]
values ≔ [45, 50, 70, 40, 30]
n ≔ 5
multiple_item_allowed ≔ истина
max_costs, item_in_knapsack ≔ knapsack(W, weights, values, n, multiple_item_allowed)
изведи("Максималната цена на раницата е: ", max_costs)
изведи("Продуктите и техният брой, които се включват в раницата са: ", item_in_knapsack)
```